

Problem Summary

Given the root of a binary tree, return true if it is height-balanced — for every node, the height difference between left and right subtrees is at most 1.

Algo

1. If root == null return 0.
2. left = height(root.left) — if -1 return -1.
3. right = height(root.right) — if -1 return -1.
4. If Math.abs(left - right) > 1 return -1.
5. Return Math.max(left, right) + 1.
6. Final answer: height(root) != -1.

Time Complexity & Space Complexity

O(n) — each node visited once, early termination on unbalanced subtrees.

O(h) — recursion stack.

Approach

Post-order recursion returning actual height for balanced subtrees and -1 as a sentinel for unbalanced ones. Early termination — once -1 is detected, propagate it up immediately without further computation.

Observations

- The -1 sentinel trick is reusable whenever you need to signal an invalid/error state through a recursive integer return.
- Order matters: check left == -1 before computing right — short circuits unnecessary work.
- Math.abs(left - right) > 1 not >= 1 — difference of exactly 1 is still balanced.

Key Insights

- -1 is the sentinel for "unbalanced" — safe because real heights are always ≥ 0.
- Early termination: once -1 is returned, all ancestors immediately return -1 too — no wasted computation.
- Brute force O(n²): call height() separately at every node. Optimal O(n): combine height + balance check in one pass.
- Checking left == -1 before computing right avoids unnecessary recursion.

Problem List

110. Balanced Binary Tree

Solved

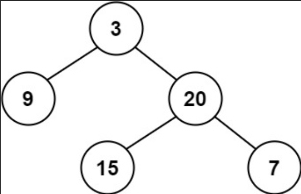
Easy

Topics

Companies

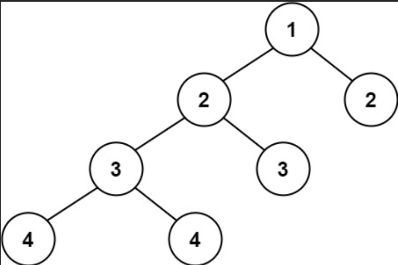
Given a binary tree, determine if it is height-balanced.

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: true

Example 2:



Accepted

228 / 228 testcases passed

Sahil Khan submitted at Jul 30, 2026 18:02

Unlock the Full LeetCode Experience

Runtime: 0 ms | Beats 100.00% | Memory: 45.48 MB | Beats 77.59%

28.28% of solutions used 1 ms of runtime

Code | Java

```
1 /**
2  * Definition for a binary tree node.
3  * public class TreeNode {
4  *     int val;
5  *     TreeNode left;
6  *     TreeNode right;
7  *     TreeNode() {}
8  *     TreeNode(int val) { this.val = val; }
9  *     TreeNode(int val, TreeNode left, TreeNode right) {
10  *         this.val = val;
11  *         this.left = left;
12  *         this.right = right;
13  *     }
14  * }
15  */
16 class Solution {
17     public boolean isBalanced(TreeNode root) {
18         return height(root) != -1;
19     }
20
21     private int height(TreeNode root){
22         if(root == null) return 0;
23
24         int left = height(root.left);
25         if(left == -1) return -1;
26
27         int right = height(root.right);
28         if(right == -1) return -1;
29
30         if(Math.abs(left-right) > 1) return -1;
31
32         return Math.max(left, right) + 1;
33     }
34 }
35
```

Testcase | Test Result

Accepted | Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input: root = [1,2,2,3,3,null,null,4,4]

Output:

12.3K

312

170 Online

3319. K-th Largest Perfect Subtree Size in Binary Tree